VE vs LE

Variable Environment vs Lexical Environment

The Two Storage Systems Inside Every Execution Context

Prepared by Claude for Akshay

1. What Are They?

Every Execution Context carries **two** internal environments:

```
Execution Context = {
  VariableEnvironment  ->  stores 'var' declarations
  LexicalEnvironment   ->  stores 'let', 'const', function, class
  this                 ->  determined by call style
}
```

> **■ The Analogy**
> **Variable Environment** = Old house with no walls between rooms — `var` roams freely (function-scoped)
> **Lexical Environment** = Modern apartment with proper walls — `let/const` stay in their room (block-scoped)

Why two? Before ES6, only `var` existed (one environment). When `let/const` were added, they needed block scoping. Instead of breaking `var`, JS added a second environment.

2. Where Each Declaration Goes

Declaration	Goes To	Hoisted As	Scope
var x = 5	VariableEnvironment	undefined	Function-scoped
let y = 10	LexicalEnvironment	(TDZ)	Block-scoped
const z = 15	LexicalEnvironment	(TDZ)	Block-scoped
function foo(){}	LexicalEnvironment	Full function body	Function-scoped
var bar = ()=>{}	VariableEnvironment	undefined	Function-scoped
class Foo {}	LexicalEnvironment	(TDZ)	Block-scoped

```
function example() {
  var a = 1;       // -> VariableEnvironment
  let b = 2;       // -> LexicalEnvironment
  const c = 3;     // -> LexicalEnvironment
  if (true) {
    var d = 4;     // -> VariableEnvironment (ignores block!)
    let e = 5;     // -> NEW LexicalEnvironment (block-scoped)
    const f = 6;   // -> NEW LexicalEnvironment (block-scoped)
  }
  console.log(a);  // 1
  console.log(d);  // 4  (var leaked out of if block!)
  console.log(e);  // ReferenceError (let stayed in block)
}
```

3. Block Behavior — The Key Difference

When entering a block (`if`, `for`, `{ }`) with `let/const`, a **NEW LexicalEnvironment** is created and chained to the previous one. The VariableEnvironment stays the **same**.

```
// BEFORE entering if block:
// VarEnv:  { x: 10, a: undefined }   <- var 'a' already here!
```

```
// LexEnv:  { y: 20 }
var x = 10; let y = 20;
if (true) {
  // NEW LexicalEnvironment created for this block!
  // VarEnv:  { x: 10, a: 40 }          <- same, a updated
  // LexEnv:  { b: 50 } -> outer -> { y: 20 }  <- NEW + chain
  var a = 40;
  let b = 50;
}
// AFTER exiting if block:
// Block LexicalEnvironment DESTROYED (b is gone!)
// VarEnv:  { x: 10, a: 40 }   <- a survived!
// LexEnv:  { y: 20 }          <- back to original
```

4. The Loop Proof — Why let Works in Loops

```
// VAR: ONE VariableEnvironment, ONE shared 'i'
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100);
}
// Output: 3, 3, 3
// VarEnv: { i: 0 -> 1 -> 2 -> 3 }  (all callbacks share this)
// LET: NEW LexicalEnvironment per iteration, OWN 'i' each time
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100);
}
// Output: 0, 1, 2
// Iteration 0: LexEnv_0 = { i: 0 }  <- callback_0 closes over this
// Iteration 1: LexEnv_1 = { i: 1 }  <- callback_1 closes over this
// Iteration 2: LexEnv_2 = { i: 2 }  <- callback_2 closes over this
```

★ This Is THE Reason

Each `let` loop iteration creates a **new LexicalEnvironment** with its own copy of `i`. Each closure captures a **different** environment. With `var`, there's only one VariableEnvironment — all closures share the same `i`.

5. Global Scope — var Goes to window, let Doesn't

```
var a = 1;      // window.a = 1     (VarEnv -> global object)
let b = 2;      // window.b = undefined! (LexEnv != global object)
const c = 3;    // window.c = undefined!
console.log(window.a);  // 1
console.log(window.b);  // undefined
console.log(window.c);  // undefined
// b and c exist in Global LexicalEnvironment
// but are NOT properties of the window object
```

■ Interview Trap

Top-level `var` becomes a property of the global object (`window`). `let` and `const` do NOT. They live in the Global LexicalEnvironment which is separate from the global object. This is a frequently asked gotcha.

6. Full Trace — Creation vs Execution Phase

```
function demo() {
  console.log(a);  // undefined   (var hoisted in VarEnv)
  console.log(b);  // ReferenceError! (let in TDZ in LexEnv)
  var a = 1;
  let b = 2;
  const c = 3;
  function greet() { return "hi"; }
}
```

```
// CREATION PHASE:
// VariableEnvironment: { a: undefined }
// LexicalEnvironment:  { b: <TDZ>, c: <TDZ>, greet: f() }
```

```
// EXECUTION PHASE (line by line):
// log(a)  -> VarEnv has a = undefined -> prints undefined
// log(b)  -> LexEnv has b = <TDZ>     -> ReferenceError!
// a = 1   -> VarEnv: { a: 1 }
// b = 2   -> LexEnv: { b: 2, c: <TDZ>, greet: f() }
// c = 3   -> LexEnv: { b: 2, c: 3, greet: f() }
```

7. Nested Scope — How Lookups Work

```
var gVar = "global-var";
let gLet = "global-let";
function outer() {
  var oVar = "outer-var";
  let oLet = "outer-let";
  function inner() {
    var iVar = "inner-var";
    let iLet = "inner-let";
    console.log(iLet);   // own LexEnv
    console.log(iVar);   // own VarEnv
    console.log(oLet);   // -> outer's LexEnv (chain)
    console.log(oVar);   // -> outer's VarEnv (chain)
    console.log(gLet);   // -> -> Global LexEnv (chain)
    console.log(gVar);   // -> -> Global VarEnv (chain)
  }
  inner();
}
```

```
// Lookup chain for inner():
//
// Looking up 'oLet':
//   inner's LexEnv? NO
//   -> outer's LexEnv? YES -> "outer-let"
//
// Looking up 'gVar':
//   inner's VarEnv? NO
//   -> outer's VarEnv? NO
//   -> Global VarEnv? YES -> "global-var"
```

8. Prove It With Chrome DevTools

```
function proof() {
  var varScoped = "I'm var";
  let letScoped = "I'm let";
  if (true) {
    var varInBlock = "var leaks";
    let letInBlock = "let stays";
    debugger;  // PAUSE 1: inside block
  }
  debugger;    // PAUSE 2: after block
}
proof();
// PAUSE 1 — Scope panel shows:
//   Block: { letInBlock: "let stays" }    <- block LexEnv
//   Local: { varScoped, varInBlock, letScoped }
// PAUSE 2 — Scope panel shows:
//   Local: { varScoped, varInBlock, letScoped }
//   letInBlock is GONE! (block LexEnv destroyed)
//   varInBlock survived! (VarEnv ignores blocks)
```

✔ **How to Test**

Open Chrome DevTools > Sources > paste the code > run. At each `debugger` pause, check the **Scope** panel on the right. You'll literally SEE the block LexicalEnvironment appear and disappear.

9. Complete Comparison

Feature	VariableEnvironment	LexicalEnvironment
Stores	var declarations only	let, const, function, class
Scope	Function-scoped	Block-scoped
Hoisting	Initialized to undefined	Hoisted but TDZ
Block { } behavior	Ignores blocks (leaks)	New env per block
In loops	ONE shared variable	NEW copy per iteration
Created when	Once per function EC	New one per block entry
Destroyed when	Function returns	Block exits
Global behavior	Becomes window.x	Does NOT become window.x
Re-declaration	Allowed (var x; var x;)	SyntaxError
Closures	One shared reference	Per-block references

10. Interview Answer & Cheat Sheet

★ Model Interview Answer

"Every Execution Context has two environments: the **VariableEnvironment** stores `var` declarations and is function-scoped, while the **LexicalEnvironment** stores `let`, `const`, and function declarations and is block-scoped. When entering a block like `if` or `for`, a new LexicalEnvironment is created and chained to the previous one, but the VariableEnvironment stays the same — which is why `var` leaks out of blocks. This is also why `let` works correctly in loops — each iteration gets its own LexicalEnvironment with its own copy of the variable."

✔ Quick Rules

1. `var` -> VariableEnvironment (function-scoped, hoisted as undefined)
2. `let/const` -> LexicalEnvironment (block-scoped, TDZ)
3. Entering a block creates a **NEW** LexicalEnvironment (chained to parent)
4. Exiting a block **DESTROYS** that LexicalEnvironment
5. VariableEnvironment **NEVER** changes on block entry/exit
6. In loops: `let` = new LexEnv per iteration, `var` = one shared VarEnv
7. Global `var` -> window property. Global `let/const` -> NOT on window
8. Use `debugger;` + Chrome Scope panel to visually prove all of this

■ One-Liner Memory Aids

VarEnv = old room, no walls, var roams free
LexEnv = modern rooms, proper walls, let/const stay put
Block entry = build a new wall (new LexEnv)
Block exit = tear down the wall (LexEnv destroyed)
var doesn't care about walls = it's in the hallway (VarEnv)